

MECHATRONICS SYSTEM INTEGRATION (MCTA 3203)

SEMESTER 2 2025/2026

WEEK 3: SERIAL COMMUNICATION BETWEEN ARDUINO AND PYTHON

SECTION 1

GROUP 9

INSTRUCTOR: DR WAHJU SEDIONO & ASSOC. PROF. EUR. ING. IR. TS. GS. INV.

DR ZULKIFLI BIN ZAINAL ABIDIN

NO	NAME	MATRIC NO
1	NOR EFFENDI BIN ANUAR @ ANWAR	2310519
2	NUR AINA SAFFIYA BINTI MOHD TARMIZI	2411644
3	NUR ATIKAH BINTI KAMARUL BAHARIN	2319846

DATE OF SUBMISSION: 25 MARCH 2026

TABLE OF CONTENTS

1.0 INTRODUCTION.....	3
2.0 MATERIALS AND EQUIPMENT.....	3
2.1 Task 1.....	3
2.2 Task 2.....	3
3.0 EXPERIMENTAL SETUP.....	4
3.1 Task 1.....	4
3.2 Task 2.....	5
4.0 METHODOLOGY.....	6
4.1 Task 1.....	6
4.1.1 Circuit Setup.....	6
4.1.2 Program Development and Uploading.....	6
4.1.3 Program Operation.....	7
4.1.4 Coding.....	7
4.2 Task 2.....	9
4.2.1 Circuit Setup.....	9
4.2.2 Program Development and Uploading.....	9
4.2.3 Program Operation.....	10
4.2.4 Coding.....	11
5.0 DATA COLLECTION.....	13
5.1 Task 1.....	13
5.2 Task 2.....	13
6.0 DATA ANALYSIS.....	14
6.1 Task 1.....	14
6.2 Task 2.....	14
7.0 RESULTS.....	16
7.1 Task 1.....	16
7.2 Task 2.....	17
8.0 DISCUSSION.....	18
9.0 CONCLUSION.....	18
10.0 RECOMMENDATIONS.....	18
11.0 REFERENCES.....	20
ACKNOWLEDGEMENT.....	21
STUDENTS' DECLARATION.....	21

1.0 INTRODUCTION

In this experiment, Python and Arduino IDE are used for bidirectional communication by establishing a two-way data exchange over a serial connection. The serial connection is used for sending the potentiometer values from Arduino to python and the python will show the value. For task 1, the bidirectional communication is used by having Arduino send the potentiometer value to the python and the python will determine when the potentiometer value exceeds the half of its maximum range then sending output signal to Arduino to light up led. Furthermore, in task 2, the potentiometer values will be used to adjust the servo motor's angle in real time by using the bidirectional communication between Arduino and Python.

The goal for this experiment is to deepen the understanding of the serial communication between Arduino and Python and using the communication for interacting or sending in input and output between the two serial communications.

2.0 MATERIALS AND EQUIPMENT

2.1 Task 1

- Microcontroller: Arduino board and USB cable
- Input Component: Potentiometer
- Output Component: LED
- Resistor: 220 Ω resistor
- Prototyping: Breadboard and Jumper wires
- Software: Arduino IDE and Python (PyCharm or IDLE)

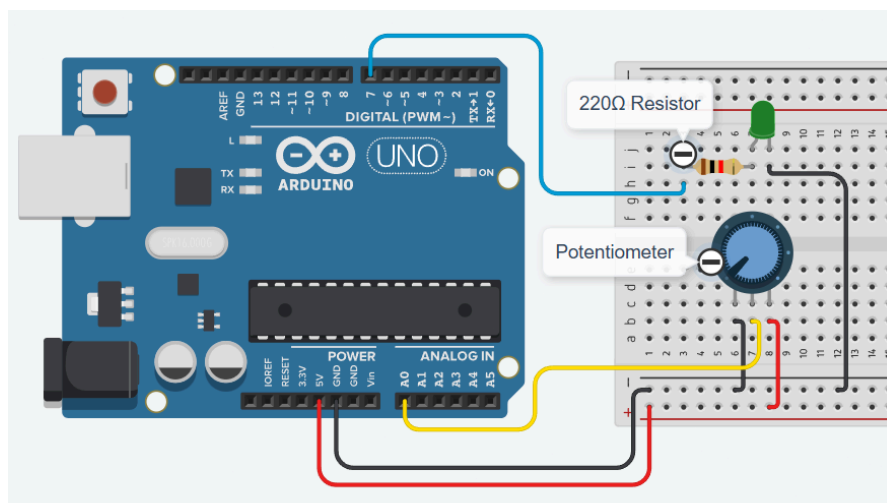
2.2 Task 2

- Microcontroller: Arduino Board and USB cable

- Actuator: Servo Motor
- Input Component: Potentiometer
- Output Component: LED
- Resistor: 220 Ω resistor
- Prototyping: Jumper wires and Breadboard
- Libraries:
 - **Arduino:** Servo.h library
 - **Python:** *pyserial* and *matplotlib*

3.0 EXPERIMENTAL SETUP

3.1 Task 1



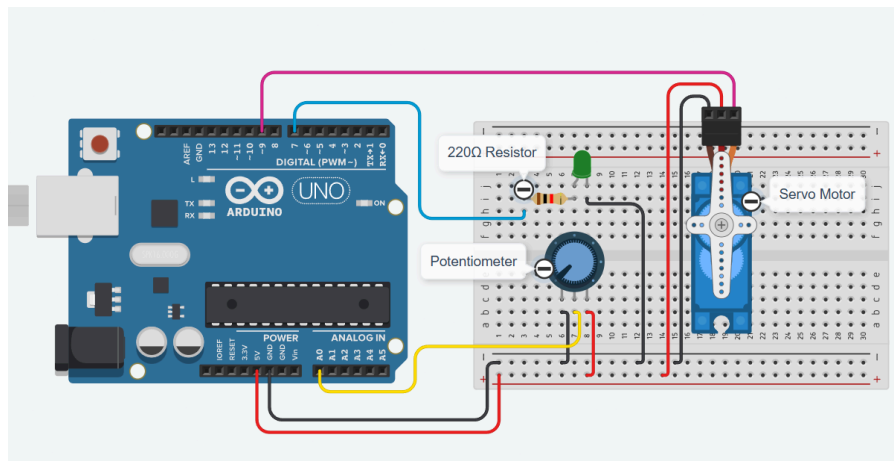
Picture 3.1 - Circuit assembly for Task 1

1. Connect the potentiometer to the Arduino Uno:
 - Connect the left potentiometer pin to the GND.
 - Connect the middle pin to the analog pin (A0).
 - Connect the right pin to the 5V pin.

2. Connect the led to the Arduino:

- Connect the cathode or negative leg of the LED to the GND pin.
- Connect the 220 Ω resistor to the anode or positive leg of the LED.
- Connect the other end of the resistor to one of the digital pins (D7).

3.2 Task 2



Picture 3.2 - Circuit assembly for Task 2

1. Connect the potentiometer to the Arduino Uno:

- Connect the left potentiometer pin to the GND.
- Connect the middle pin to the analog pin (A0).
- Connect the right pin to the 5V pin.

2. Connect the led to the Arduino:

- Connect the cathode or negative leg of the LED to the GND pin.
- Connect the 220 Ω resistor to the anode or positive leg of the LED.
- Connect the other end of the resistor to one of the digital pins (D7).

3. Connect the servo to the Arduino:

- Connect the brown wire to the GND pin.

- Connect the red wire to the 5V pin.
- Connect the orange wire to one of the digital pins (D9).

4.0 METHODOLOGY

4.1 Task 1

4.1.1 Circuit Setup

- Components required to be prepared are Arduino Uno, potentiometer, 220 Ω resistor, LED, breadboard and wire jumpers for assembling the circuit based on the Picture 3.1.
- The potentiometer is connected to the Arduino pins which are 5V and analog pin A0. The GND potentiometer pin is connected to the GND terminal on the breadboard.
- The LED anode leg is connected to the 220 Ω resistor, and the resistor will be connected to the digital pin D2.
- The LED cathode leg is connected to the GND terminal on the breadboard which is connected to the Arduino GND pin.
- After finishing the circuit assembly, the wiring is checked carefully to ensure the circuit will work when turned on and to prevent short circuits from happening.

4.1.2 Program Development and Uploading

- The Arduino Integrated Development Environment (IDE) is opened on the computer.
- A new sketch is used to write a program to send the input which is

potentiometer value to Python and receive command from Python to turn on or off the LED through serial communication.

- The program is verified to check if there's no error and upload it to Arduino Uno.
- The Pycharm software is opened on the computer.
- A new project is opened to write a program to receive the input value from Arduino, making conditions on when the LED will turn on or off and send the output signal to Arduino to turn on or off the LED.

4.1.3 Program Operation

- The pin that connects to the anode LED is configured as output pin while the pin that connects to the potentiometer is configured as input pin.
- The Arduino is monitoring the input from the potentiometer to send to Python and output signal from Python to interact with LED.
- The Python monitors the input value given from Arduino and processes the command based on the value to send the output signal to Arduino.
- The potentiometer will give input value to the Arduino and Python, and it will affect the LED brightness where the low input value will make the LED dimmer and high input value will make the LED light up very bright.

4.1.4 Coding

- Arduino Code:

```
int potPin = A0;
int ledPin = 2;

void setup() {
    Serial.begin(9600);
    pinMode(ledPin, OUTPUT);
}

void loop() {
```

```

int val = analogRead(potPin);
Serial.println(val);

if (Serial.available() > 0) {
  char command = Serial.read();
  if (command == '1') {
    digitalWrite(ledPin, HIGH);
  } else if (command == '0') {
    digitalWrite(ledPin, LOW);
  }
}
delay(100);
}

```

- **Python Code:**

```

import serial
import time

# Change 'COMx' according to your own port
ser = serial.Serial('COM5', 9600)
time.sleep(2)

try:
    while True:
        if ser.in_waiting > 0:
            line = ser.readline().decode('utf-8').strip()
            if line.isdigit():
                val = int(line)
                print(f"Reading: {val}")

                if val > 512:
                    ser.write(b'1') # Send '1' to ON
                else:
                    ser.write(b'0') # send '0' to OFF
            time.sleep(0.1)
except KeyboardInterrupt:
    ser.close()

```


4.2 Task 2

4.2.1 Circuit Setup

- Components required to be prepared are Arduino Uno, potentiometer, 220 Ω resistor, LED, servo motor, breadboard and wire jumpers for assembling the circuit based on the Picture 3.2.
- The potentiometer is connected to the Arduino pins which are 5V and analog pin A0. The GND potentiometer pin is connected to the GND terminal on the breadboard.
- The LED anode leg is connected to the 220 Ω resistor, and the resistor will be connected to the digital pin D13. The LED cathode leg is connected to the GND terminal on the breadboard which is connected to the Arduino GND pin.
- The servo motor red wire is connected to 5V terminal, the brown wire is connected to GND terminal and the orange wire is connected to digital pin D9.
- After finishing the circuit assembly, the wiring is checked carefully to ensure the circuit will work when turned on and to prevent short circuits from happening.

4.2.2 Program Development and Uploading

- The Arduino Integrated Development Environment (IDE) is opened on the computer.
- A new sketch is used to write a program to send the input which is potentiometer value to Python and receive command from Python to turn on or off the LED and move the servo according to the potentiometer value through serial communication.
- The program is verified to check if there's no error and upload it to Arduino

Uno.

- The Pycharm software is opened on the computer.
- A new project is opened to write a program to receive the input value from Arduino, making conditions on when the LED will turn on or off, the servo move according to the potentiometer value and sends the output signal to Arduino to turn on or off the LED and move the servo.
- The Python also will make a graph of the potentiometer resistor value where it shows the current value of the potentiometer in real time when interacting with the potentiometer.

4.2.3 Program Operation

- The digital pins that connect to the anode LED and the orange wire servo motor are configured as output pins while the analog pin that connects to the potentiometer is configured as input pin.
- The Arduino is monitoring the input from the potentiometer to send to Python and output signal from Python to interact with the LED and servo motor.
- Python monitors the input value given from Arduino and processes the command based on the value to send the output signal to Arduino.
- The potentiometer will give input value to the Arduino and Python, and it will affect the LED brightness where the low input value will make the LED dimmer and high input value will make the LED light up very bright. The value will also affect the movement of the servo motor from 0° to 180° or 180° to 0°.

4.2.4 Coding

- **Arduino Code:**

```
#include <Servo.h>

Servo myServo;
const int potPin = A0;
const int ledPin = 13;
const int servoPin = 9;

void setup() {
  Serial.begin(9600);
  myServo.attach(servoPin);
  pinMode(ledPin, OUTPUT);
}

void loop() {
  int potValue = analogRead(potPin);
  int angle = map(potValue, 0, 1023, 0, 180);

  myServo.write(angle);

  if (potValue > 512) digitalWrite(ledPin, HIGH);
  else digitalWrite(ledPin, LOW);

  // Send value to phyton
  Serial.println(potValue);

  delay(50);
}
```

- **Python Code:**

```
import sys
import time

sys.path.append(r'C:\\Program
Files\\Python314\\Lib\\site-packages')

import serial
import matplotlib.pyplot as plt

PORT = 'COM5'

try:
  ser = serial.Serial(PORT, 9600, timeout=1)
  time.sleep(2)
  print(f"CONNECTED TO PORT")
```

```

except:
    print("Cant connected to port")
    sys.exit()

plt.ion()
fig, ax = plt.subplots()
data_list = [0] * 50
line, = ax.plot(data_list, 'b-', linewidth=2) # Blue
line

ax.set_ylim(0, 1023)
ax.set_title("Task 2: Real-Time Plot")
ax.grid(True)

try:
    while True:
        line_raw =
        ser.readline().decode('utf-8').strip()
        if line_raw:
            try:
                val = int(line_raw)
                data_list.append(val)
                data_list.pop(0)
                line.set_ydata(data_list)
                plt.pause(0.01)
            except ValueError:
                pass
except KeyboardInterrupt:
    ser.close()
    plt.close()

```

5.0 DATA COLLECTION

5.1 Task 1

Potentiometer Position	ADC Value (0-1023)	LED Status (ON/OFF)
Min (Fully Left)	0	OFF
25% Turn	256	OFF
50% (Midpoint)	512	OFF
75% Turn	768	ON
Max (Fully Right)	1023	ON

Table 5.1 shows the status of the LED when the potentiometer is rotated.

5.2 Task 2

Potentiometer Input (ADC)	Mapped Servo Angle (°)	LED Status (ON/OFF)
0	0	OFF
256	45	OFF
512	90	OFF
768	135	ON
1023	180	ON

Table 5.2 shows the status of the LED and servo motor angle when the potentiometer is rotated.

6.0 DATA ANALYSIS

6.1 Task 1

Based on the results obtained in Task 1, the relationship between the potentiometer position and the ADC value shows a linear behavior. When the potentiometer is rotated from the minimum position (fully left) to the maximum position (fully right), the ADC value increases gradually from 0 to 1023. Intermediate positions such as 25%, 50%, and 75% correspond to ADC values of approximately 256, 512, and 768, respectively. This indicates that the analog input is being correctly converted into digital values by the microcontroller's ADC.

In terms of LED behavior, the LED remains OFF from ADC values 0 to 512, and only turns ON when the ADC value exceeds approximately 768. This suggests that a threshold condition has been implemented in the system. The LED is likely programmed to activate only when the input voltage (represented by the ADC value) surpasses a certain level, which corresponds to around 75% of the potentiometer rotation.

This behavior demonstrates that the system is functioning correctly as a basic comparator, where the LED acts as an indicator when the input exceeds a predefined limit. It also confirms that the potentiometer is effectively controlling the analog voltage input, and the ADC is accurately translating it into digital form.

6.2 Task 2

Based on the results obtained in Task 2, a clear and consistent relationship can be observed between the potentiometer input (ADC value) and the mapped servo angle. The ADC values, which range from 0 to 1023, are linearly mapped to servo angles between 0° and 180°. For instance, ADC values of 256, 512, and 768 correspond to servo positions of approximately

45°, 90°, and 135°, respectively. This demonstrates that the mapping function is accurately implemented, allowing proportional control of the servo motor based on the potentiometer's rotation.

The linearity of this relationship indicates that the system provides smooth and predictable motion, where any small change in the potentiometer position results in a corresponding change in the servo angle. This is important in practical applications, as it ensures precise control over the actuator without sudden jumps or instability.

In addition to servo control, the LED behavior further enhances the system's functionality. The LED remains OFF at lower ADC values and turns ON when the ADC value reaches 768 and above. This suggests that a predefined threshold has been set, where the LED acts as an indicator to signal that the potentiometer has reached a higher range of operation. The consistency of this behavior with Task 1 confirms that the threshold condition is reliably implemented.

Overall, the system successfully integrates analog input, signal processing, and output response. The potentiometer effectively controls the servo motor position in a linear manner, while the LED provides a clear visual indication once a specific input level is exceeded. These results demonstrate that the system is functioning as intended, with accurate mapping and stable performance.

7.0 RESULTS

7.1 Task 1

The result from this experiment is the LED is turned on when the value of potentiometer reaches half of its maximum. This shows that the serial data transmission from arduino to python is successful. The arduino successfully read the value of potentiometer and transmitted to the python via USB serial communication. Diagram 7.1.1 is the connection of the circuit. The potentiometer is connected in Analog pin in the arduino while led is connected in digital in the Arduino. After that, the terminal of the Phyton shows the real time value of the potentiometer. The value displayed changed as the potentiometer knob turned. Furthermore, the red LED is turned on when the value of potentiometer exceeds half of its value. It shows that the LED successfully integrated in the circuit.

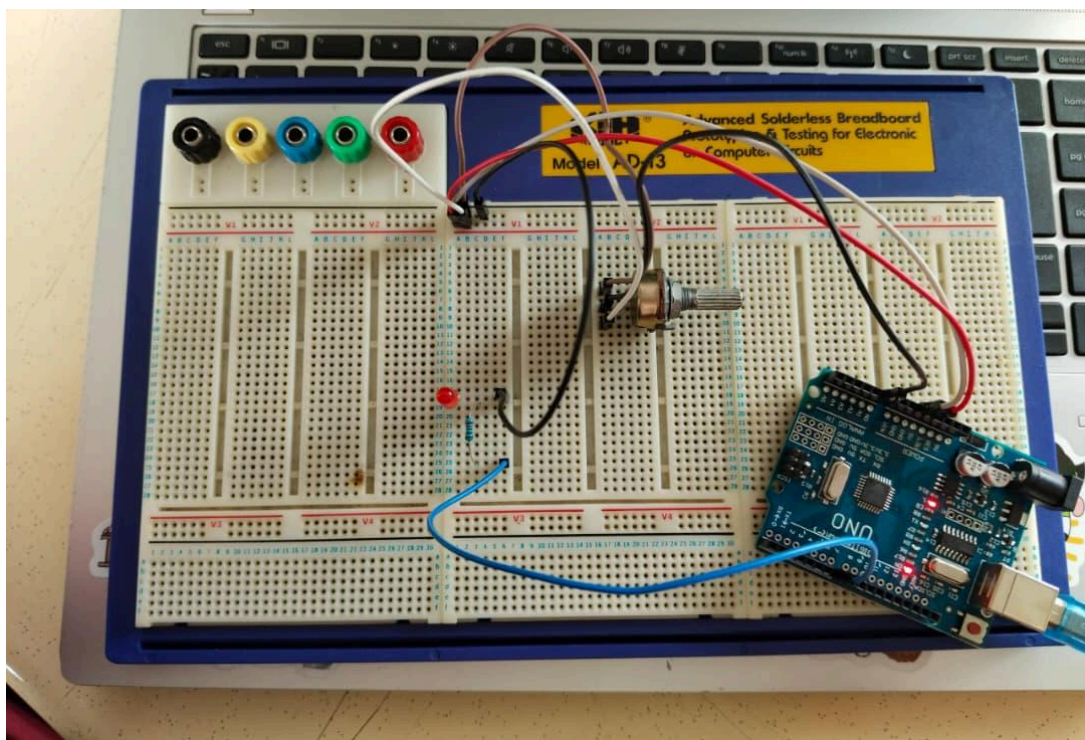


Figure 7.1.1 show the circuit of the experiment in task 1

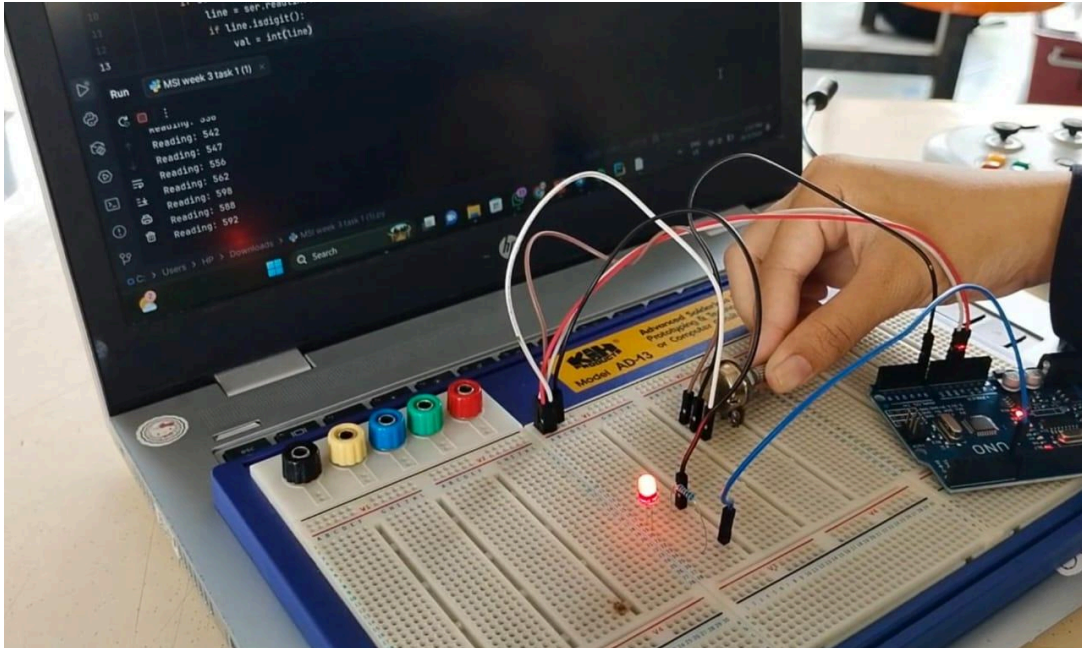


Figure 7.1.2 show the LED turn on

7.2 Task 2

In this experiment, the servo angle successfully turned in a specific angle according to the movement of the potentiometer knob. This shows that the system is using real time adjustment and the servo motor can be controlled using potentiometer. After that, we used matplotlib lib in this experiment to see the real time potentiometer value. The dynamic graph was successfully generated using the python. This experiment also integrated with the experiment in task 1. From this we can know that the value of the potentiometer is exceed half and the angle of the servo motor is over 90 degrees.

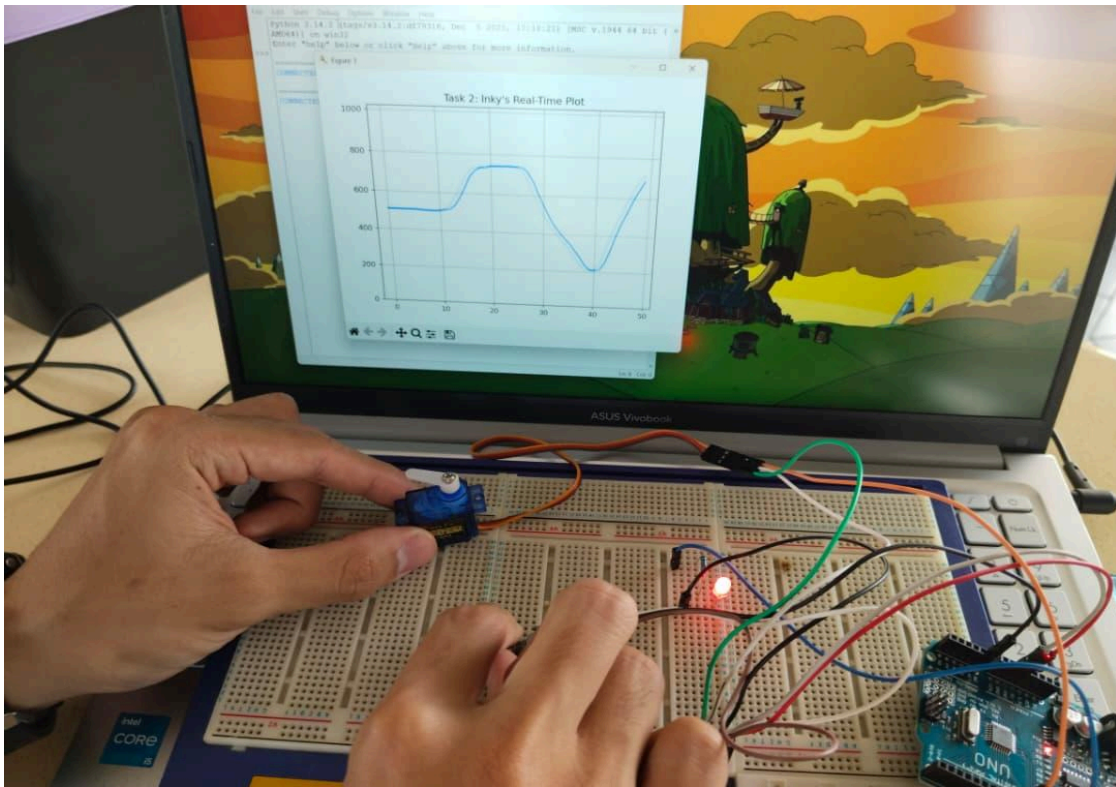


Diagram 7.2.1 show the real time plot graph and servo motor moving

8.0 DISCUSSION

8.1 Task 1

The objective of this experiment is to make a serial communication between arduino and Phyton. The critical thing to alert in this experiment is to make sure the baud rate in the coding for both arduino and python is the same. For example the baud rate we use in this task is 9600. If the baud rate is not the same, both of them can not communicate with each other. After that, we also need to make sure to turn off the serial monitor or serial plotter in Arduino IDE before running the code in python. This is because both of them need to access the serial port and only one can use the serial port at a time. Other things we get from this experiment is that the A0 pin turns the voltage from potentiometer to 10 bit digital value.

8.2 Task 2

In this experiment, we focussed on controlling the servo motor. To make it happen, we need to use a library like “<servo.h>” to control the servo motor in PWM pin 9 and use “myservo.write()” to control the real time angle of the servo motor. This experiment also highlights the communication between Arduino and Phyton. To make them able to communicate we need to encode the data from Phyton using “.encode()” before sending it to Arduino. On the other hand, the data from the arduino need to be decoded and stripped. The example of the coding is “.decode().strip()”.

9.0 CONCLUSION

In conclusion, the experiments successfully demonstrated the integration of Arduino and Python to create a responsive mechatronics system through serial communication. By completing Task 1, bidirectional control was established, allowing the Arduino to send potentiometer data to Python while simultaneously receiving commands to trigger an LED. In Task 2, the system's complexity was increased to include mechanical actuation and data visualization, where a servo motor was controlled in real-time using both terminal inputs and potentiometer mapping. The use of Python's *matplotlib* library allowed for the transformation of raw serial data into a live graphical plot, proving that high-level programming can effectively monitor hardware performance. Ultimately, these tasks confirmed that combining a microcontroller's hardware interface with Python's processing power enables sophisticated monitoring and automated control of integrated systems.

10.0 RECOMMENDATIONS

One of the recommendations for this week's tasks is to save the potentiometer value to excel file format. Since a graph is used to monitor real time potentiometer resistor value when the potentiometer is interacted, it does not save the data for future usage. So, it would be better to save and collect the input value for data collection for future reference or analysis.

Furthermore, ThinkerCad can be used to do the simulation without the risk of components getting broken. The LED can only light up for a while with full voltage (5V) before it slowly breaks and becomes unusable and the lowest resistor value from potentiometer makes the LED light up very bright which increases the risk of being broken. So, having a digital simulation to simulate the experiment or task is better to prevent any risk of broken components.

11.0 REFERENCES

Last Minute Engineers. (2019, October 20). How Servo Motor Works & Interface It With Arduino. Last Minute Engineers.

<https://lastminuteengineers.com/servo-motor-arduino-tutorial/>

Tutorials | Arduino Documentation. (n.d.). Docs.arduino.cc. <https://docs.arduino.cc/tutorials/>

Instructables. (2015, July 10). Arduino Servo Motors. Instructables; Instructables.

<https://www.instructables.com/Arduino-Servo-Motors/>

ACKNOWLEDGEMENT

We would like to sincerely express our gratitudes to our instructors, Assoc. Prof. Eur. Ing. Ir. Ts. Gs. Inv. Dr. Zulkifli Bin Zainal and the lab technician, for the help given during the task. The knowledge and suggestions are able to guide us throughout the progress of the task. We also would like to thank our classmates for their support and help which really give positive impacts to the progression of our task.

STUDENTS' DECLARATION

We hereby declare that the work presented in this technical report is entirely our own, except where explicitly acknowledged. We certify that in developing and completing this mechatronics project, we have strictly adhered to the standards of academic integrity and have not engaged in any form of plagiarism or unethical conduct. All sources of information, reference materials, and technical support utilized in this work have been appropriately cited and acknowledged.

Certificate of Originality and Authenticity

This is to certify that we are responsible for the work submitted in this report, that the original work is our own except as specified in the references and acknowledgment, and that the original work contained herein has not been untaken or done by unspecified sources or persons. We hereby certify that this report has not been done by only one individual, and all of us have contributed to the report. The length of contribution to the reports by each individual is noted within this certificate. We also hereby certify that we have read and understand the content of the total report, and no further improvement on the reports is needed from any of the individual contributors to the report. We therefore agreed

unanimously that this report shall be submitted for marking, and this final printed report has been verified by us.

Signature: 

Read [/]

Name: NOR EFFENDI BIN ANUAR @ ANWAR

Understand [/]

Matric Number: 2310519

Agree [/]

Signature: 

Read [/]

Name: NUR AINA SAFFIYA BINTI MOHD TARMIZI

Understand [/]

Matric Number: 2411644

Agree [/]

Signature: 

Read [/]

Name: NUR ATIKAH BINTI KAMARUL BAHARIN

Understand [/]

Matric Number: 2319846

Agree [/]